SI100B: Introduction to Information Science and Technology

Lecture 6

By Ana Bell of MIT

FUNCTIONS as ARGUMENTS

HIGHER ORDER PROCEDURES

- Objects in Python have a type
  - int, float, str, Boolean, NoneType, function
- Objects can appear in RHS of assignment statement
  - Bind a name to an object
- Objects
  - Can be **used as an argument** to a procedure
  - Can be **returned as a value** from a procedure
- Functions are also **first class objects**!
- Treat functions just like the other types
  - Functions can be bound to new names
  - Functions can be arguments to another function
  - Functions can be returned by another function

OBJECTS IN A PROGRAM

```
def is_even(i):
    return i%2 == 0

r = 2

pi = 22/7
```

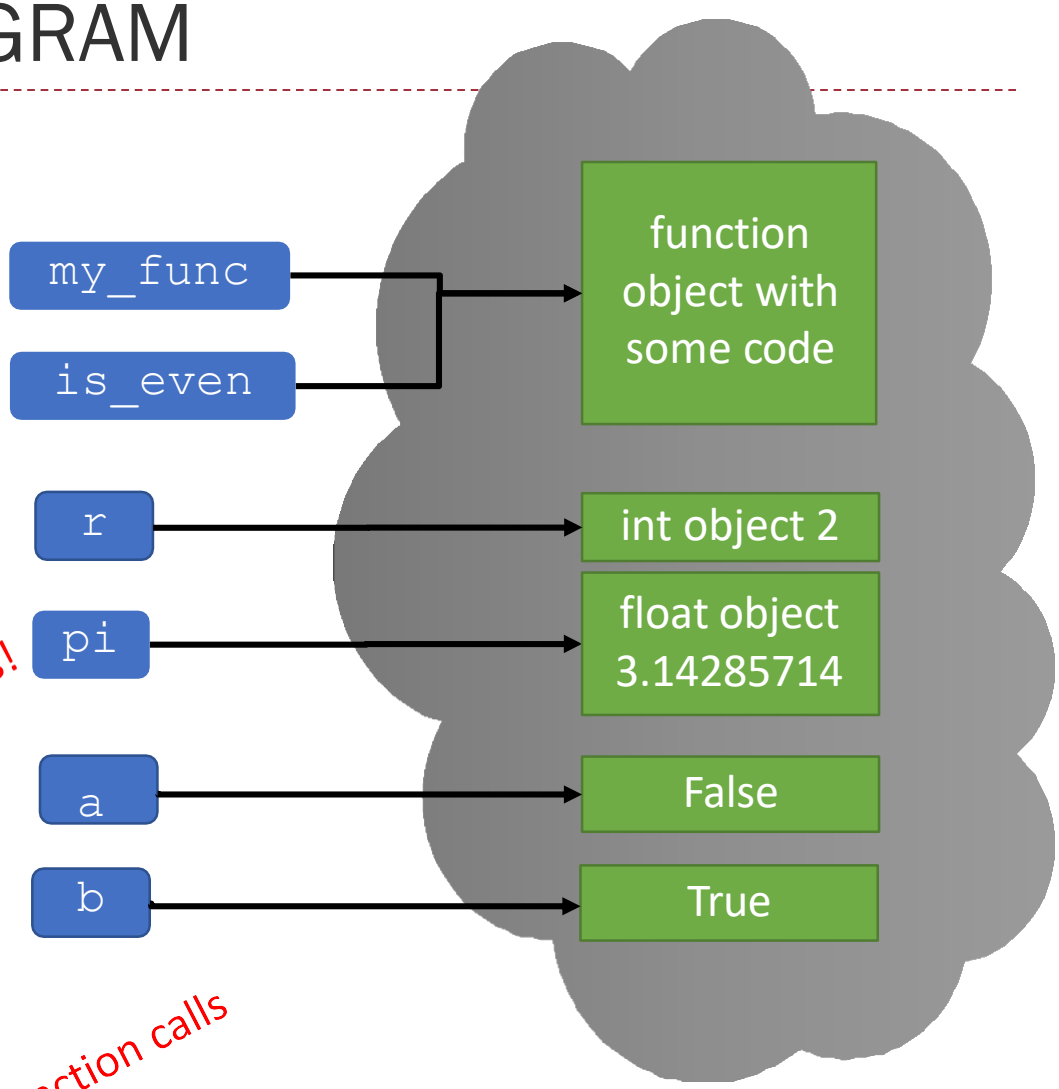
```
my_func = is_even
```

```
a = is_even(3)
```

```
b = my_func(4)
```

NOT a function call, just names!

Two function calls



BIG IDEA

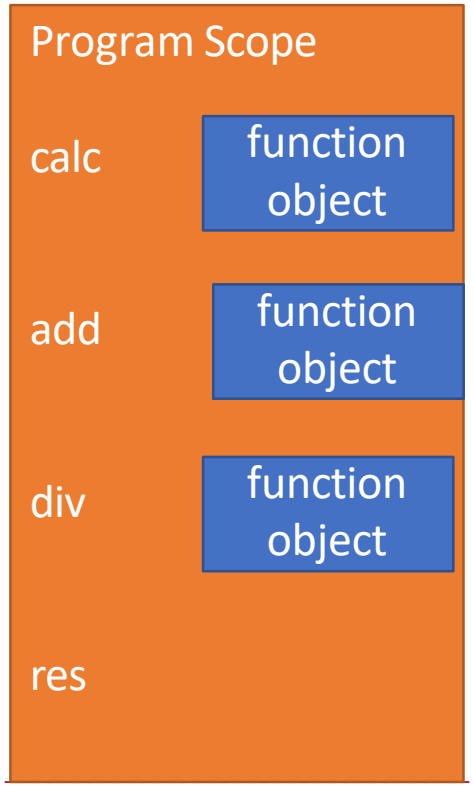
- Everything in Python is an object.

FUNCTION AS A PARAMETER

```
def calc(op, x, y):
    return op(x,y)


def add(a,b):
    return a+b


def div(a,b):
    if b != 0:
      return a/b
    print("Denominator was 0.")


print(calc(add, 2, 3))
```

STEP THROUGH THE CODE

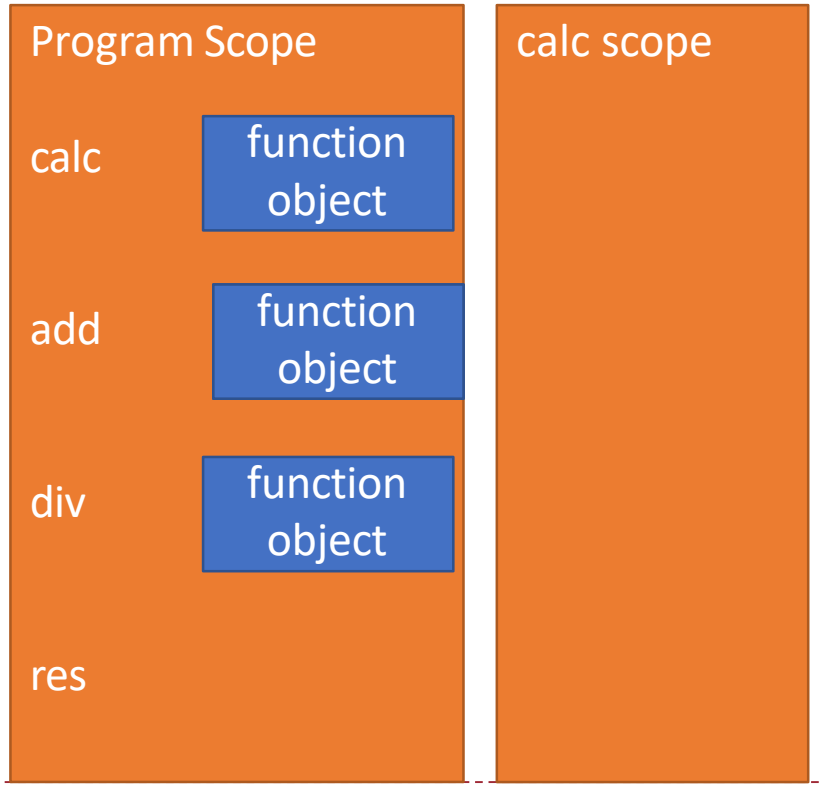
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```



CREATE calc SCOPE

```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

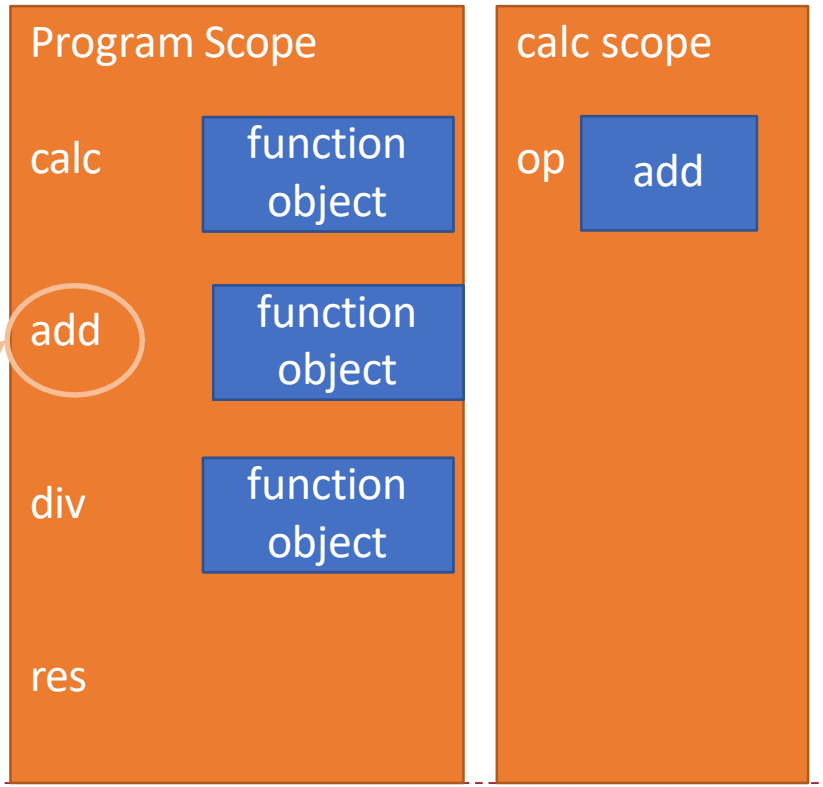
Function call



MATCH FORMAL PARAMS in calc

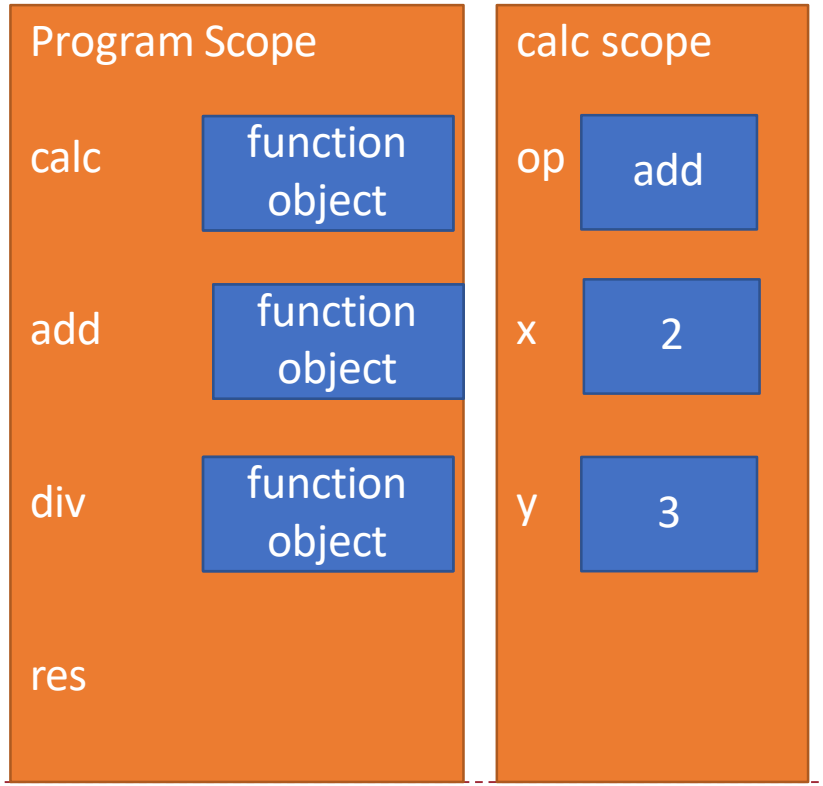
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

function obj NOT
function call



MATCH FORMAL PARAMS in calc

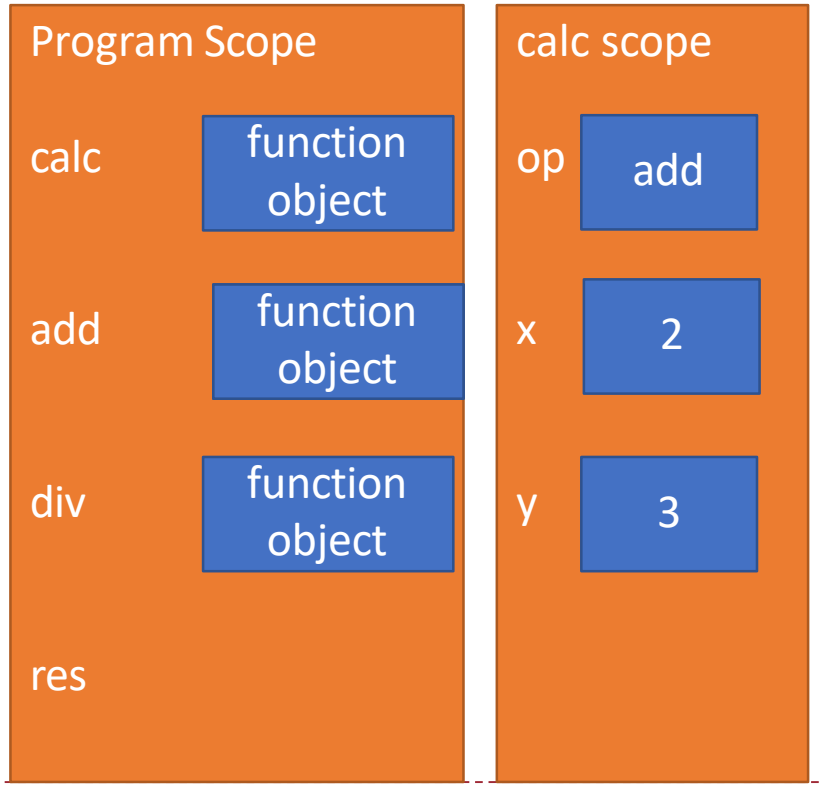
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```



FIRST (and only) LINE IN calc

```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

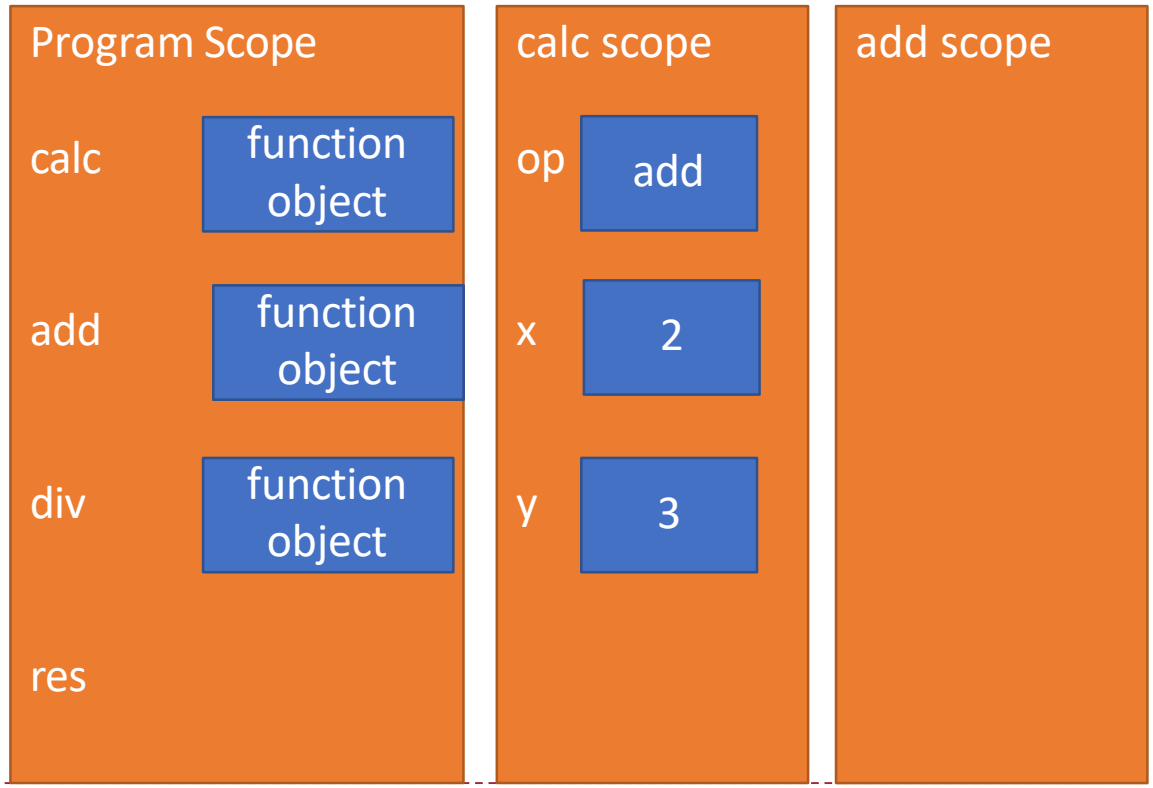
return add(2,3)
Just replace each param with its value



CREATE SCOPE OF add

```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

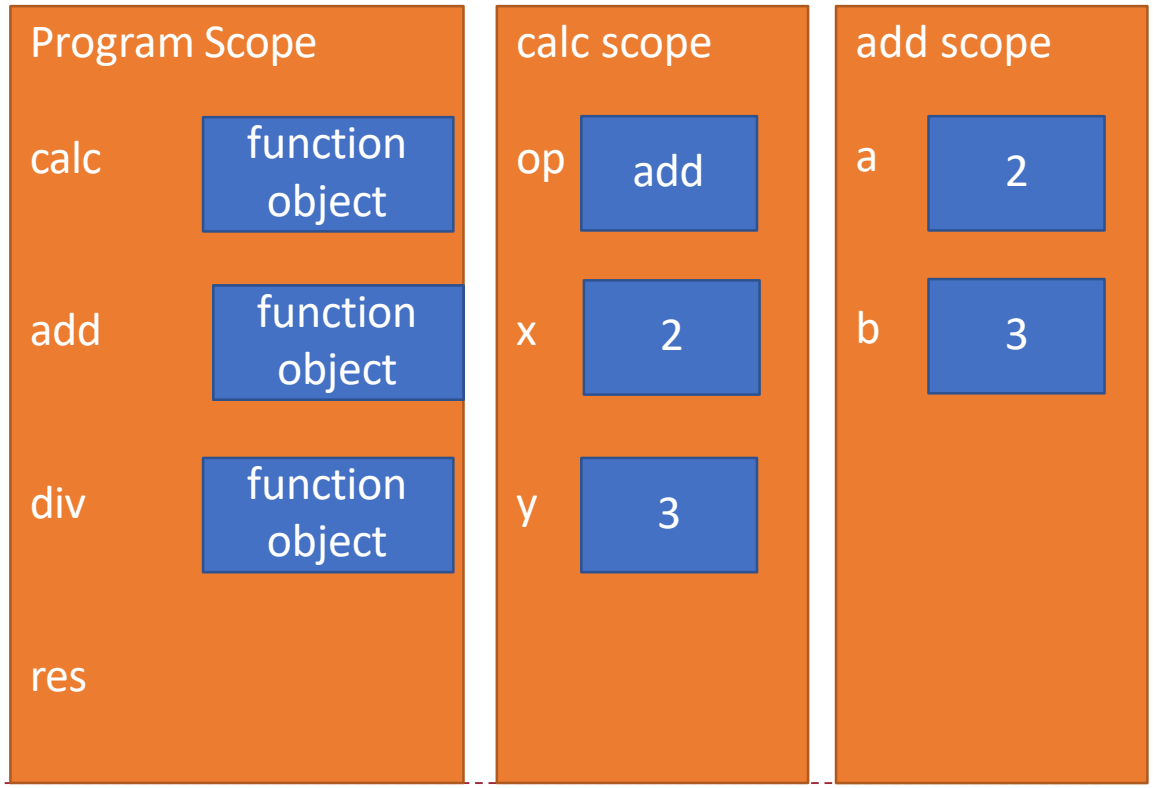
Function call in calc scope: add(2,3)



MATCH FORMAL PARAMS IN add

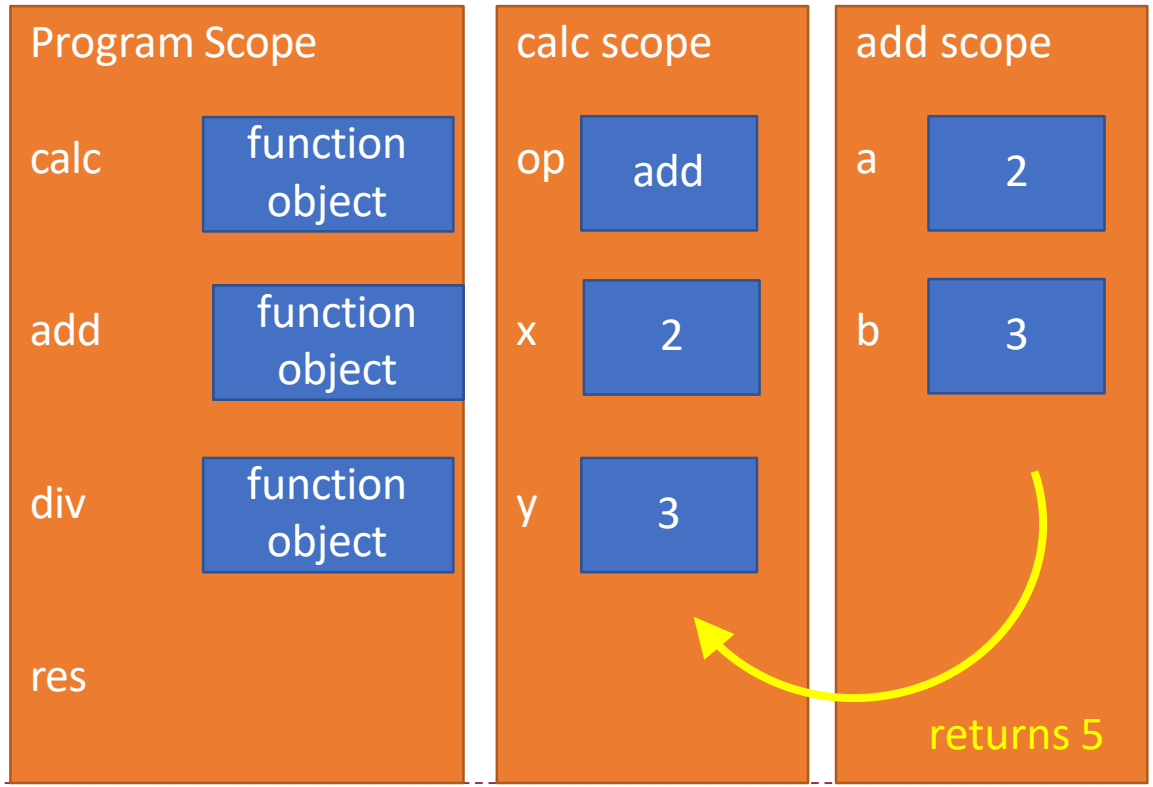
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

Function call in calc scope: add with formal params a=2 and b=3



EXECUTE LINE OF add

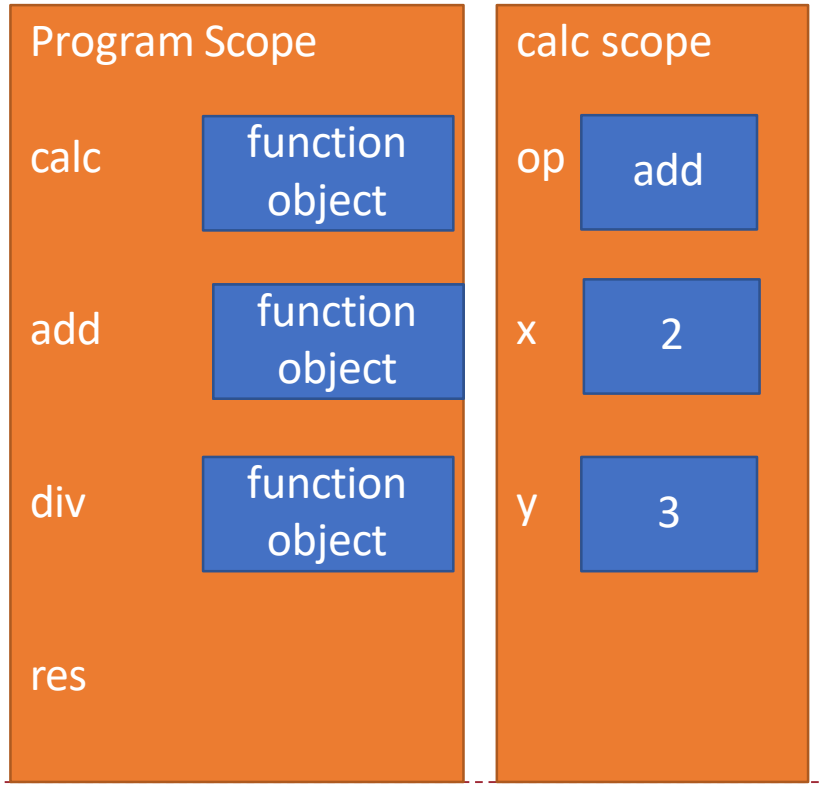
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```



REPLACE FUNC CALL WITH RETURN

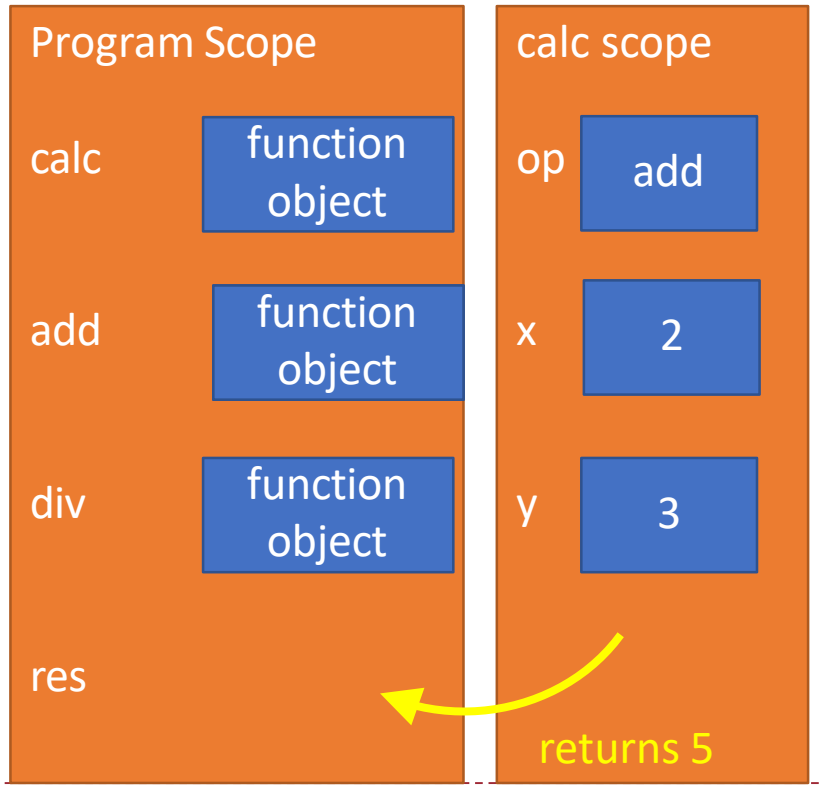
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

5



EXECUTE LINE OF calc

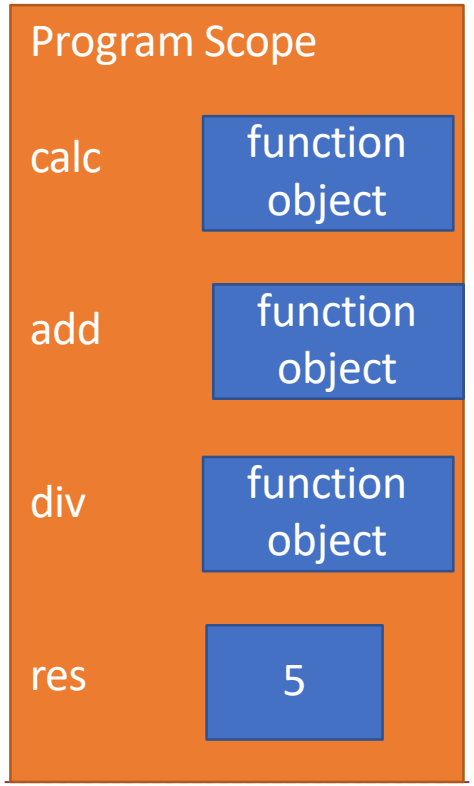
```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```



REPLACE FUNC CALL WITH RETURN

```
def calc(op, x, y):
    return op(x,y)

def add(a,b):
    return a+b

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(add, 2, 3)
```

5



YOU TRY IT!

- Do a similar trace with the function call

```
def calc(op, x, y):
    return op(x,y)

def div(a,b):
    if b != 0:
        return a/b
    print("Denom was 0.")

res = calc(div,2,0)
```

- What is the value of res and what gets printed?

ANOTHER EXAMPLE: FUNCTIONS AS PARAMS

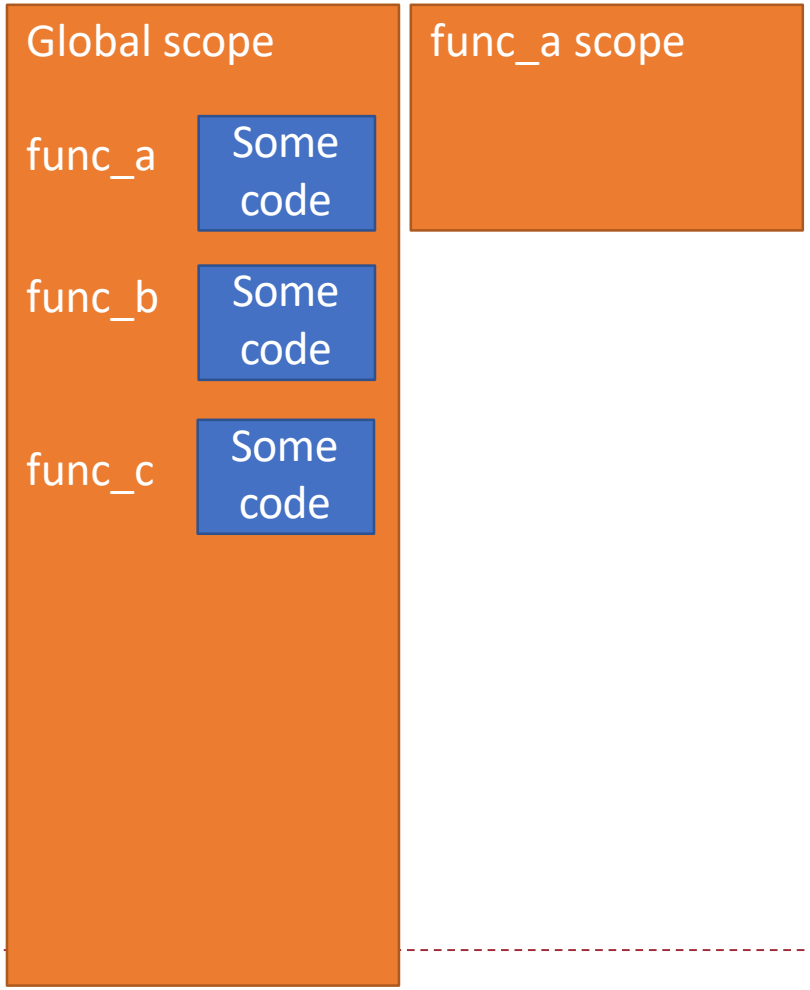
```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```

call `func_a`, takes no parameters
call `func_b`, takes one parameter, an int
call `func_c`, takes two parameters, another function and an int

FUNCTIONS AS PARAMETERS

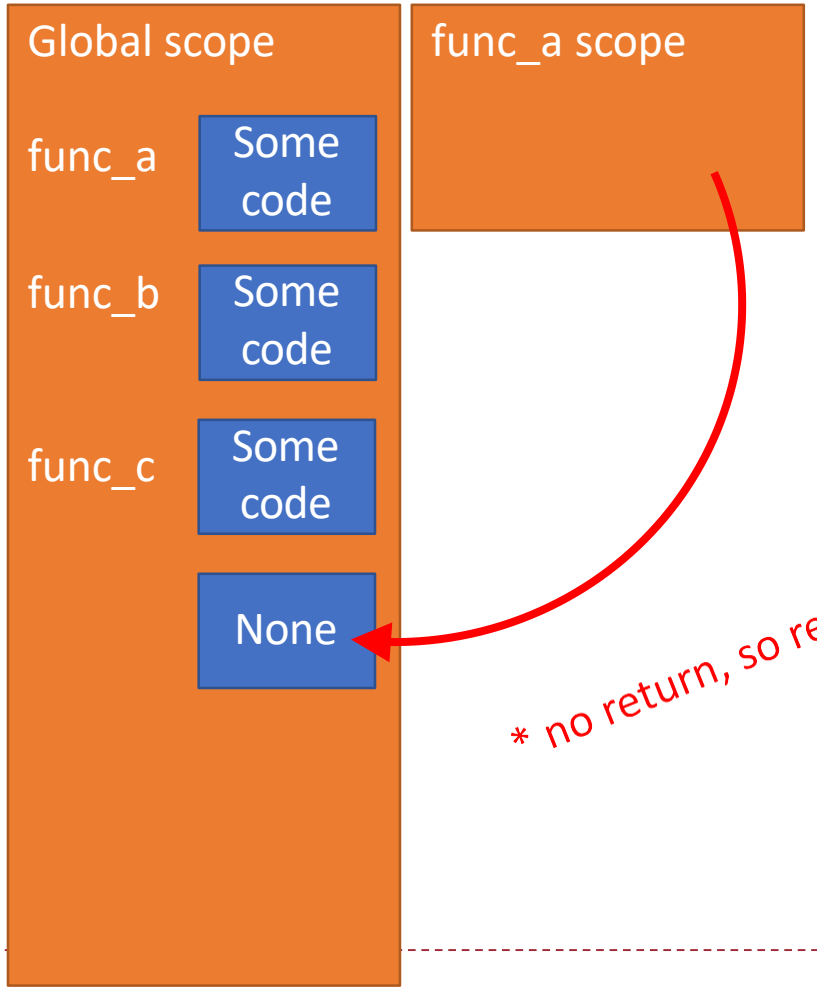
No bindings (no parameters)

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```



FUNCTIONS AS PARAMETERS

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```

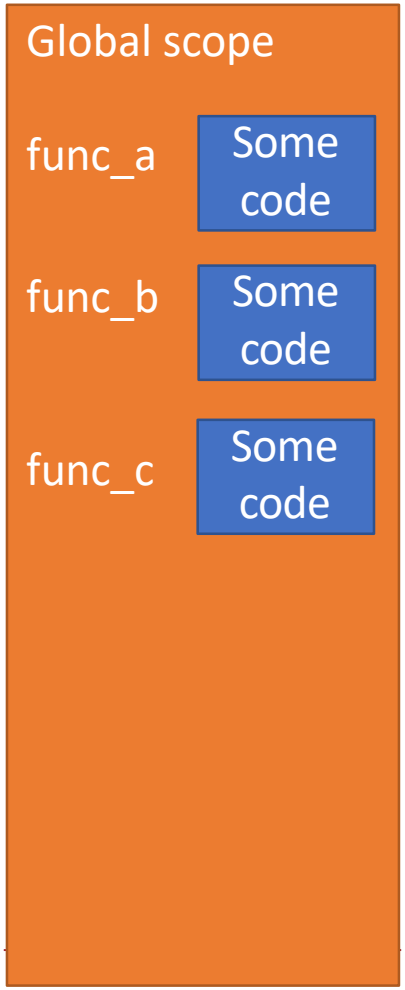


body prints 'inside func_a' on console

* no return, so returns None

FUNCTIONS AS PARAMETERS

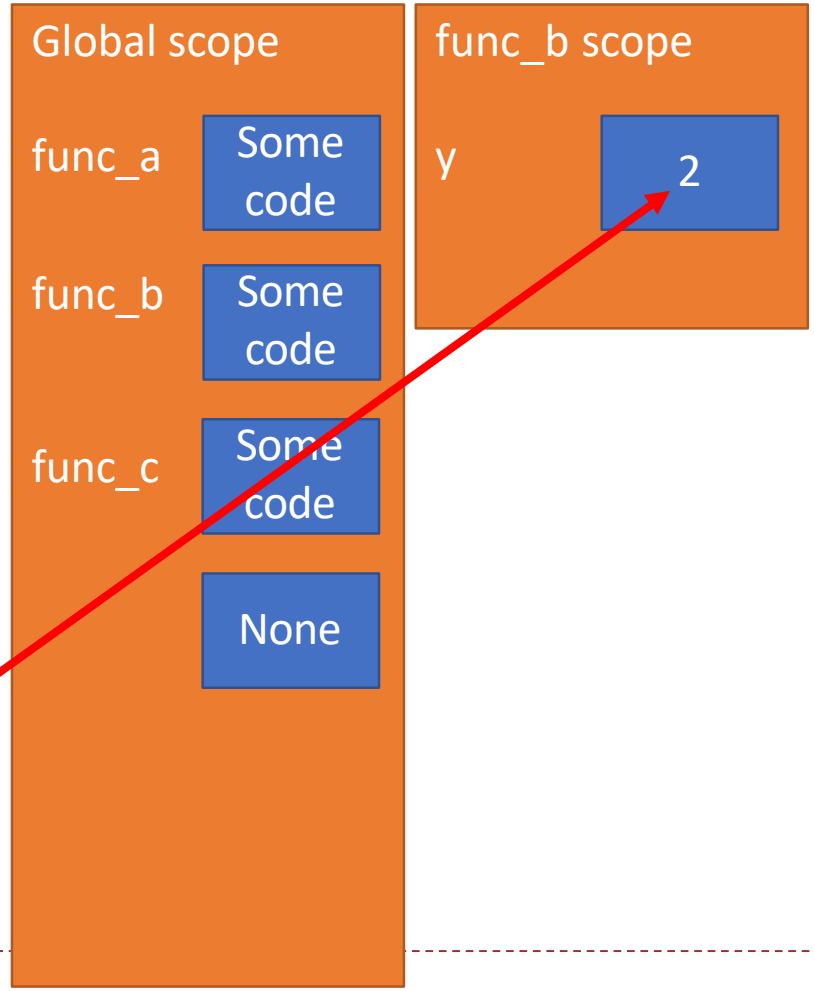
```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```



print displays None on console

FUNCTIONS AS PARAMETERS

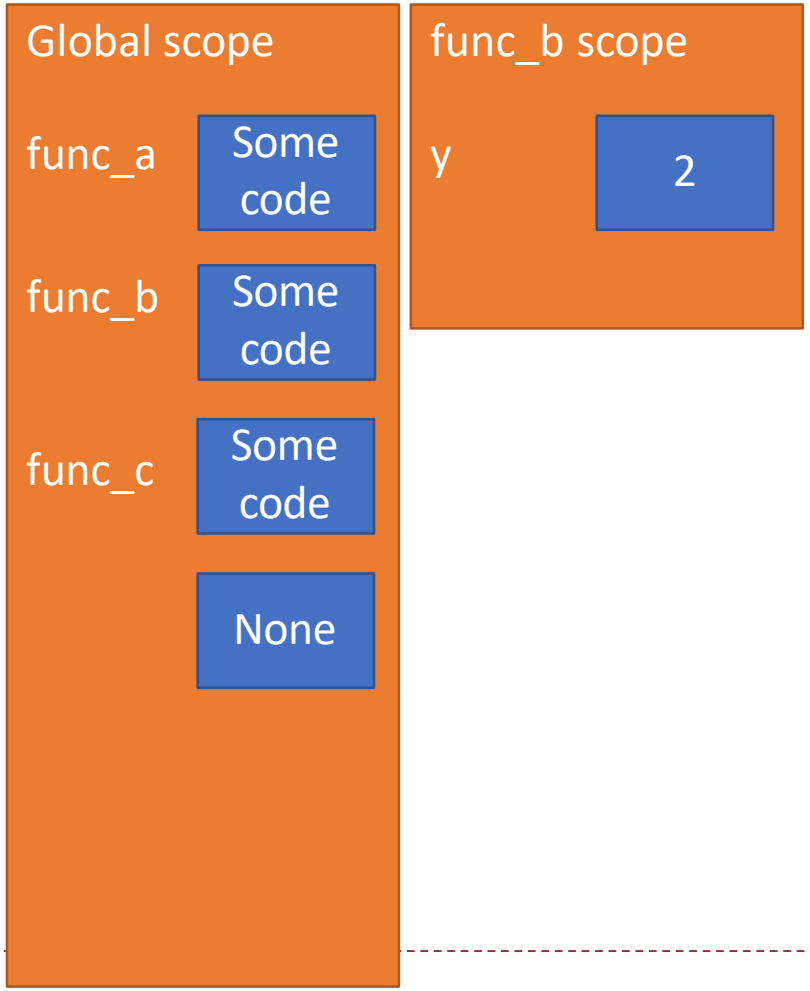
```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```



FUNCTIONS AS PARAMETERS

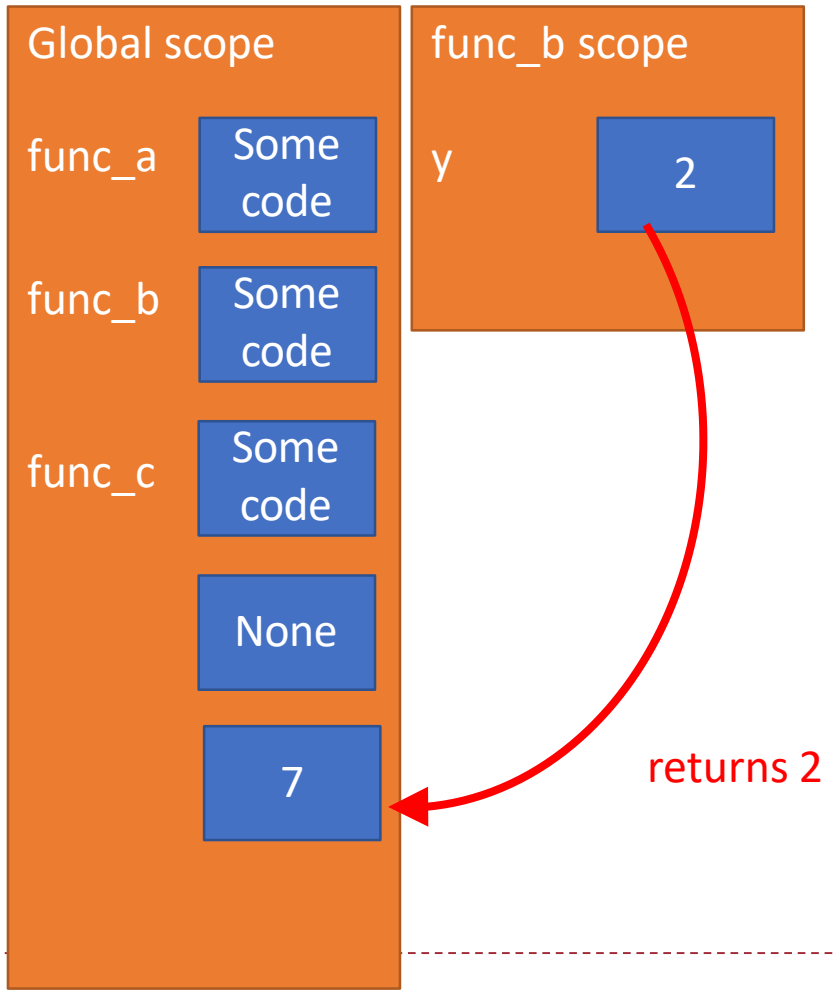
body prints 'inside func_b' on console

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```



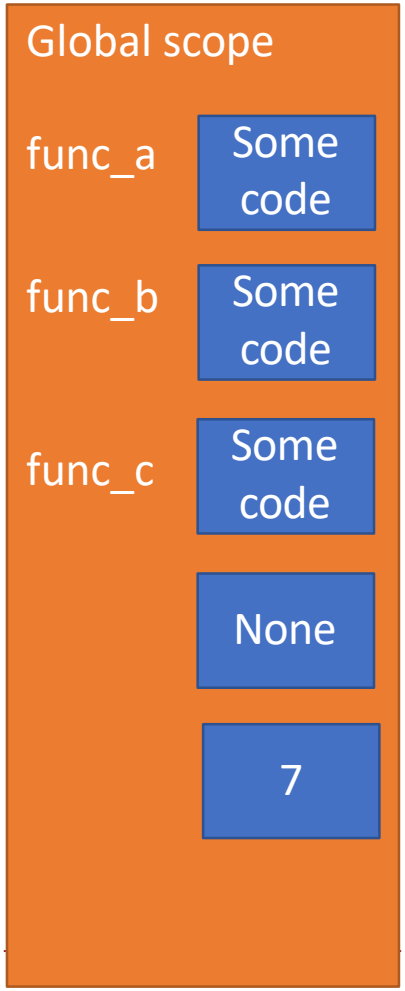
FUNCTIONS AS PARAMETERS

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```



FUNCTIONS AS PARAMETERS

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```

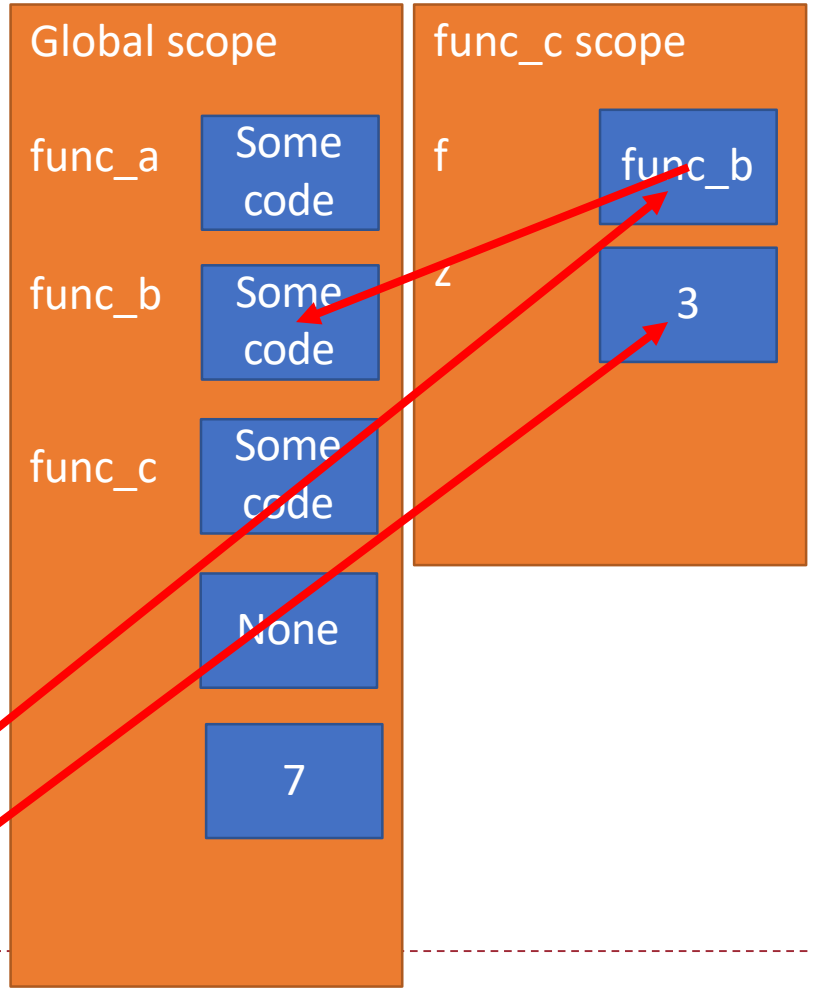


print displays 7 on console

FUNCTIONS AS PARAMETERS

body of func_c prints 'inside func_c' on console

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```

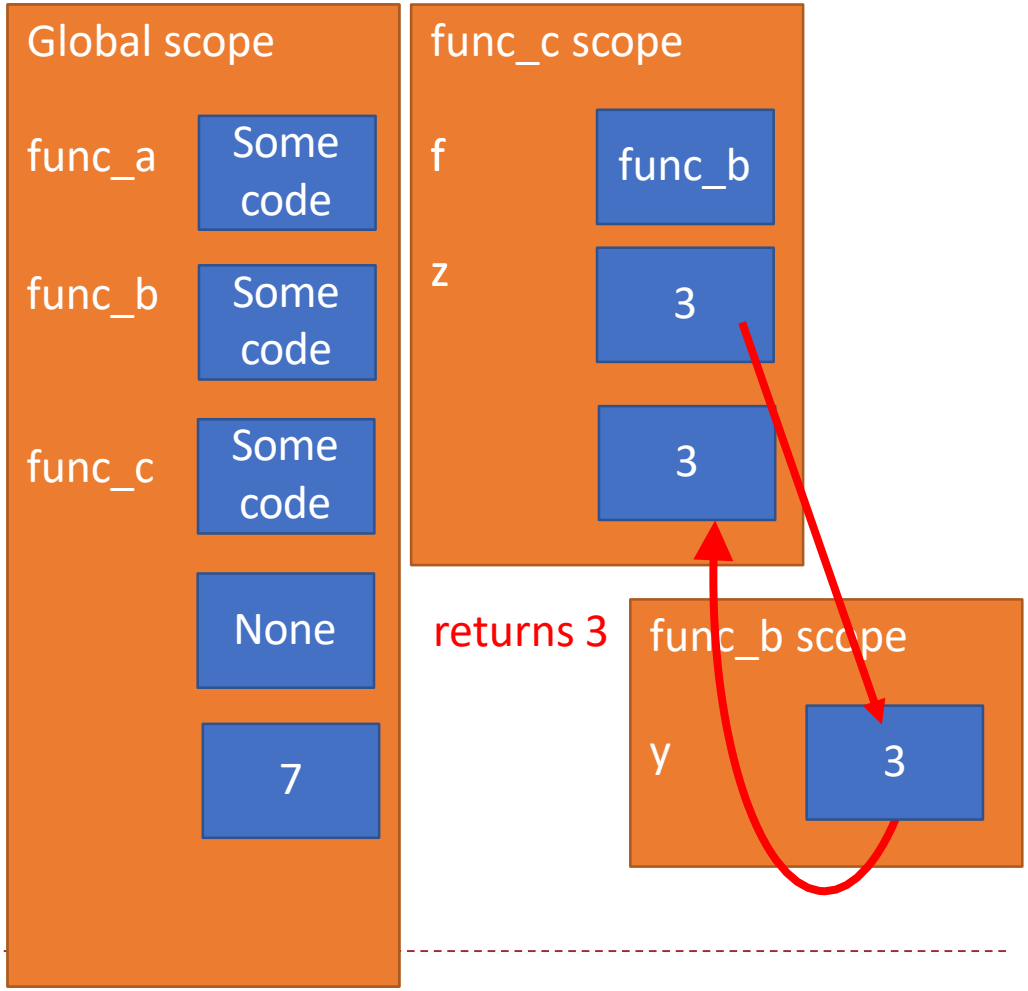


FUNCTIONS AS PARAMETERS

body of func_b prints 'inside func_b' on console

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```

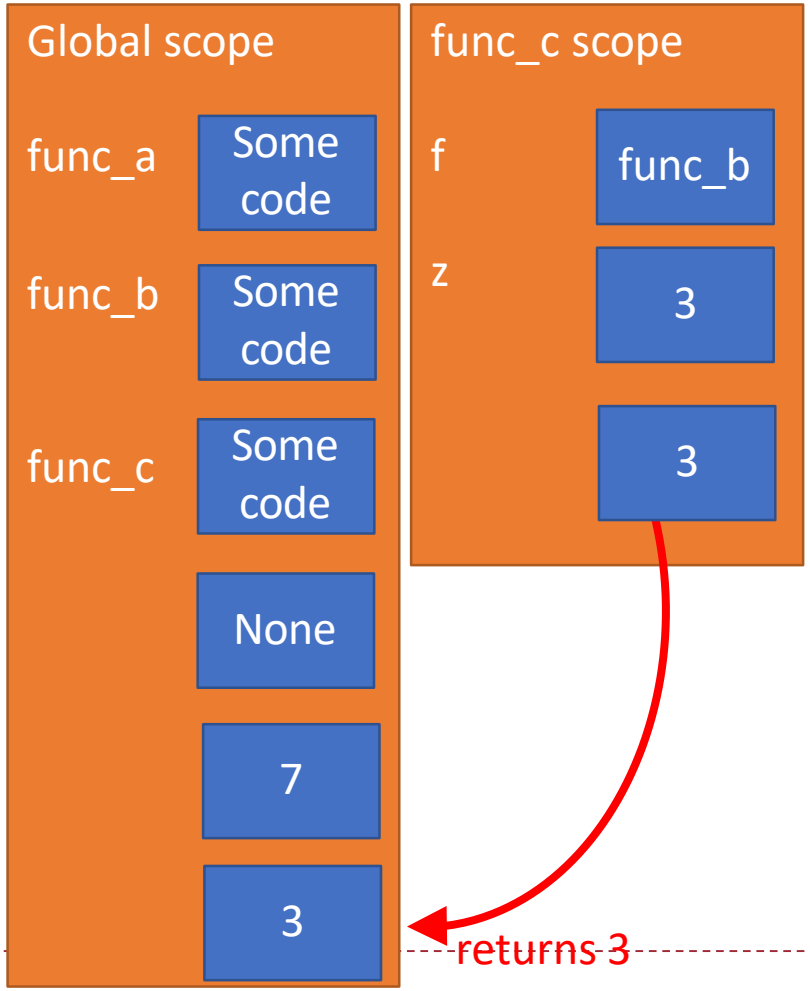
3



FUNCTIONS AS PARAMETERS

print displays 3 on console

```
def func_a():
    print('inside func_a')
def func_b(y):
    print('inside func_b')
    return y
def func_c(f, z):
    print('inside func_c')
    return f(z)
print(func_a())
print(5 + func_b(2))
print(func_c(func_b, 3))
```



YOU TRY IT!

- Write a function that meets these specs.

```
def apply(criteria,n):
    """
    criteria is a func that takes in a number and returns a bool
    n is an int
    Returns how many ints from 0 to n (inclusive) match the
    criteria (i.e. return True when run with criteria)
    """

def is_even(x):
    return x % 2 == 0

print(apply(is_even, 10))
```

FUNCTIONS RETURNING FUNCTIONS

FUNCTIONS CAN RETURN FUNCTIONS

```
def make_prod(a):
    def g(b):
        return a*b
    return g
```

This function def is inside another function.

This is NOT a function call!

```
val = make_prod(2)(3)
print(val)
```

SAME

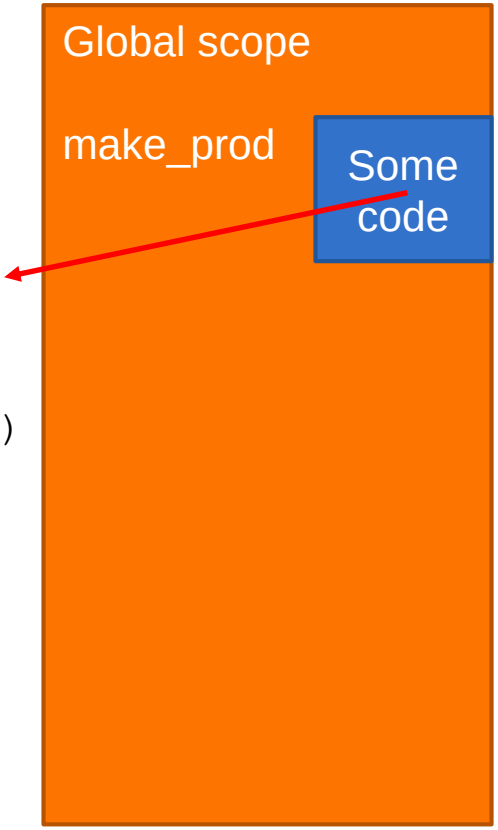
```
doubler = make_prod(2)
val = doubler(3)
print(val)
```

SCOPE DETAILS FOR WAY 1

```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```

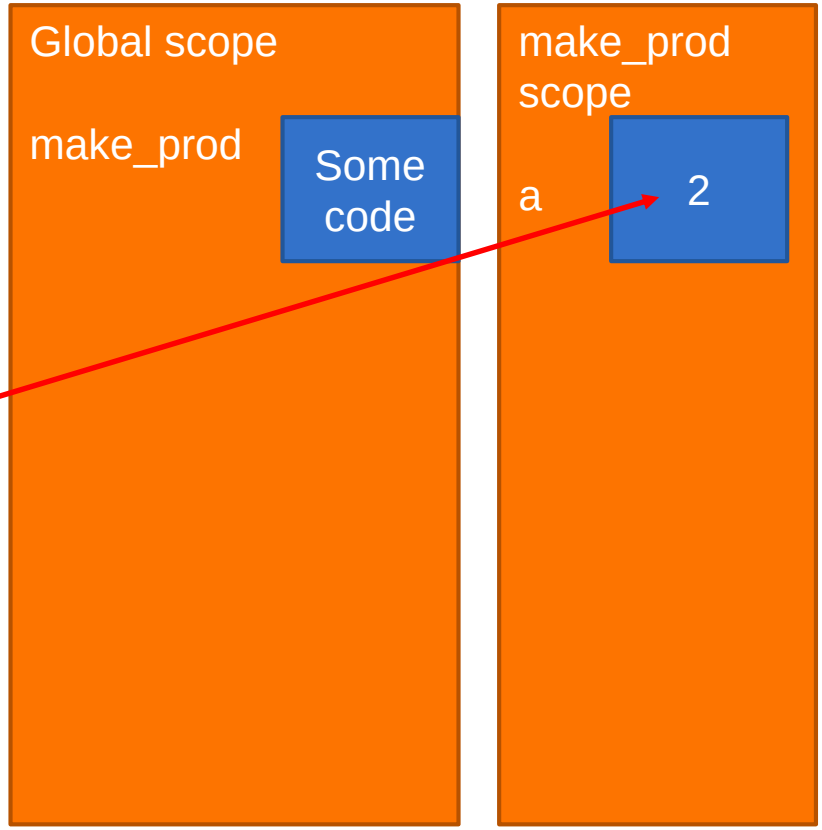
SCOPE DETAILS FOR WAY 1

```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```



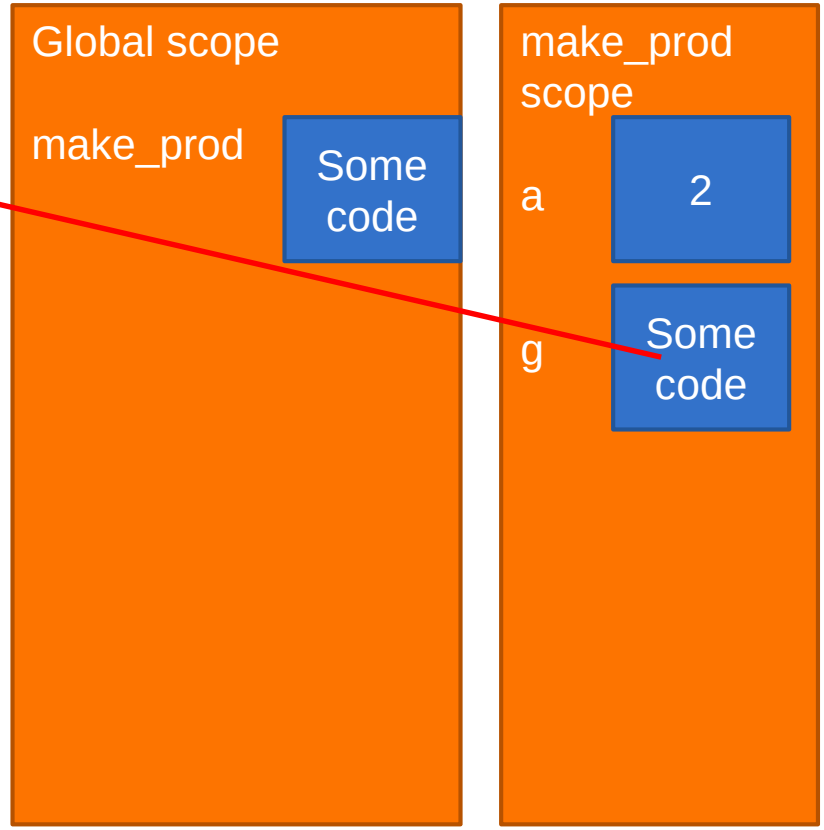
SCOPE DETAILS FOR WAY 1

```
def make_prod(a):
    def g(b):
        return a*b
    return g


val = make_prod(2)(3)
print(val)
```



SCOPE DETAILS FOR WAY 1

```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```



NOTE: definition of g is done within scope of make_prod, so binding of g is within that frame/scope

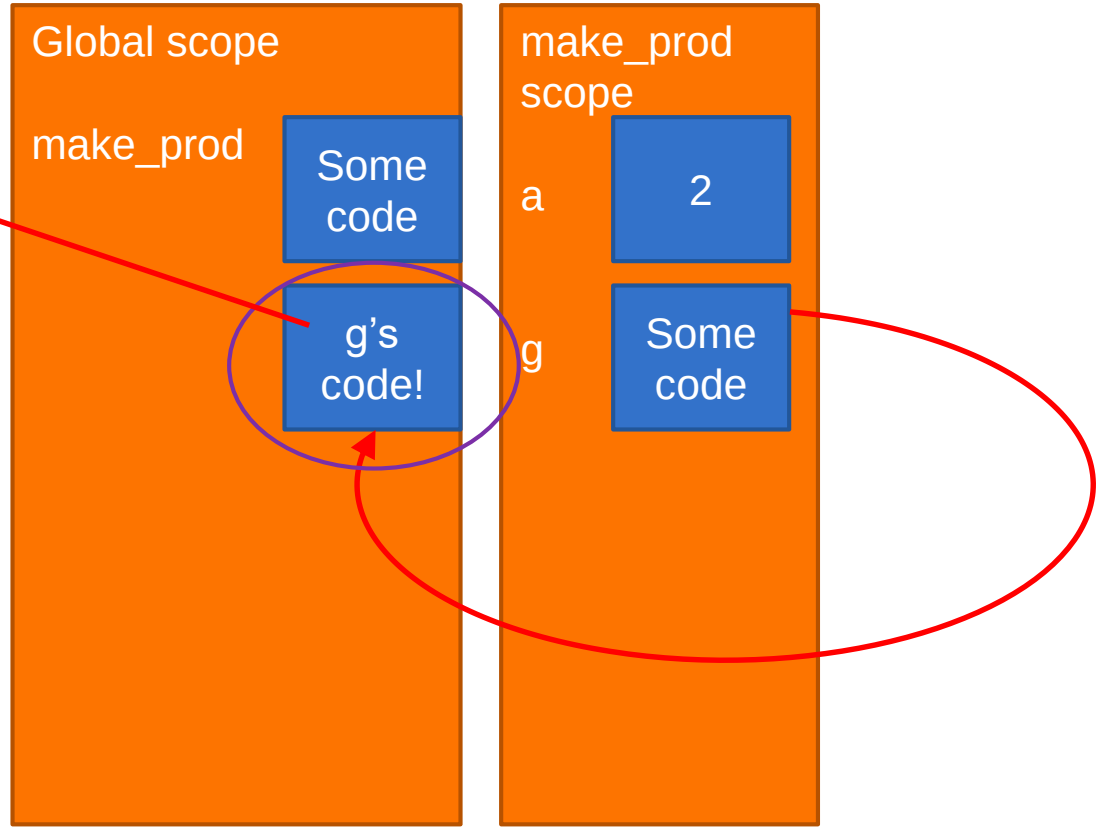
Since g is bound in this frame, cannot access it by evaluation in global frame

g can only be accessed within call to make_prod, and each call will create a new, internal g

SCOPE DETAILS FOR WAY 1

```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```

This is g



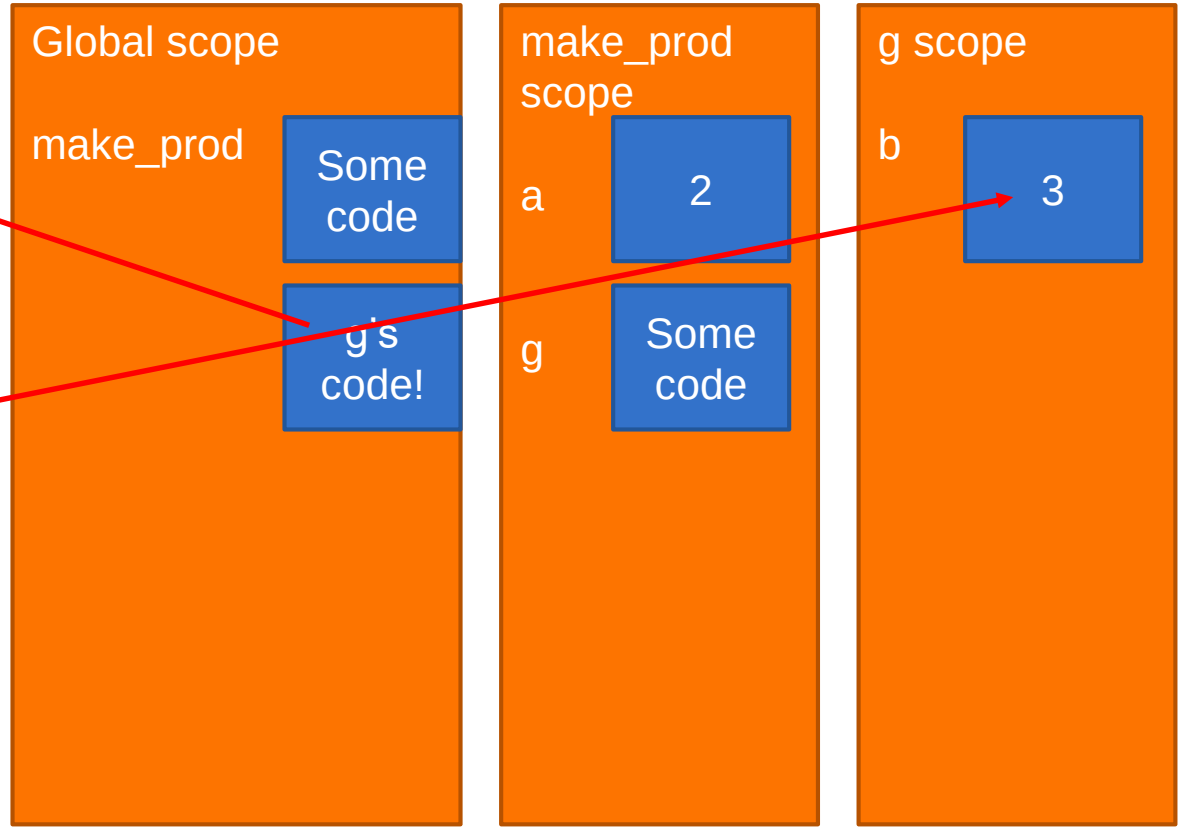
Evaluating `make_prod(2)` has returned an anonymous procedure

Returns pointer to g code

SCOPE DETAILS FOR WAY 1

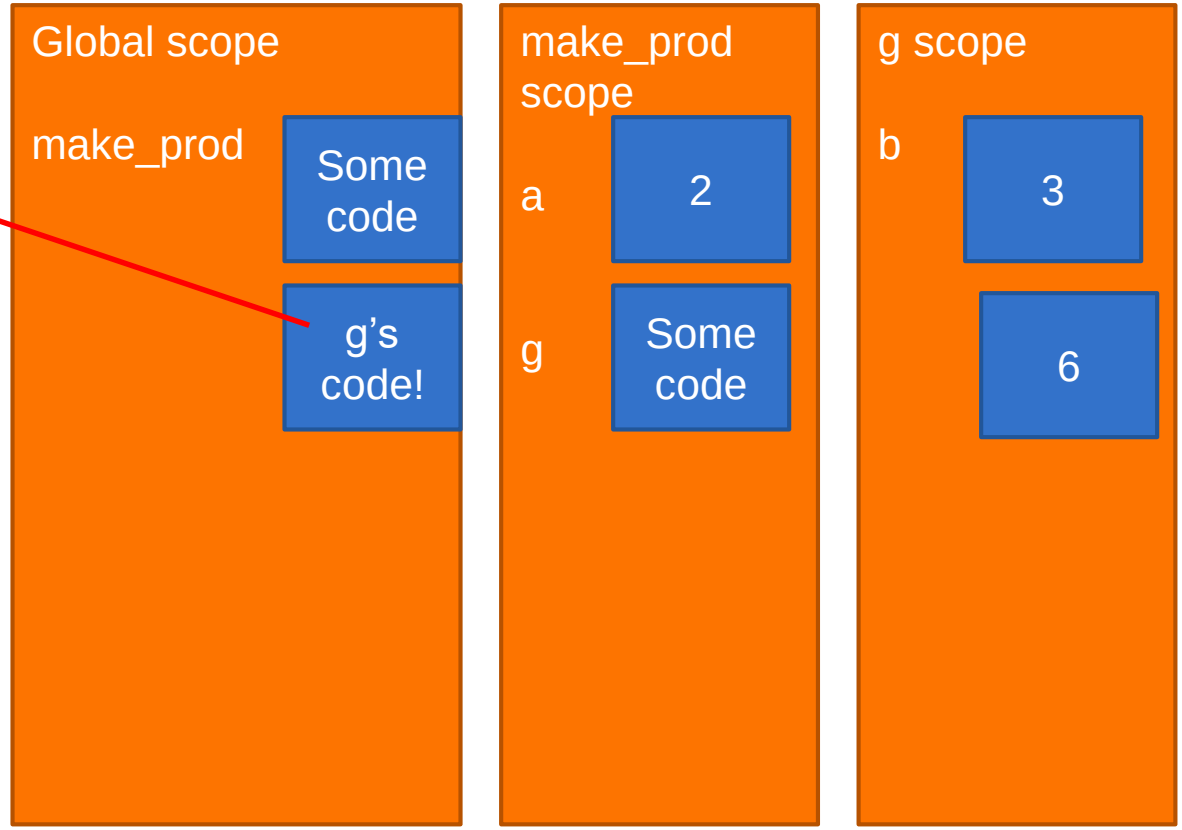
```
def make_prod(a):
    def g(b):
        return a*b
    return g


val = make_prod(2)(3)
print(val)
```

Call is g(3)



SCOPE DETAILS FOR WAY 1

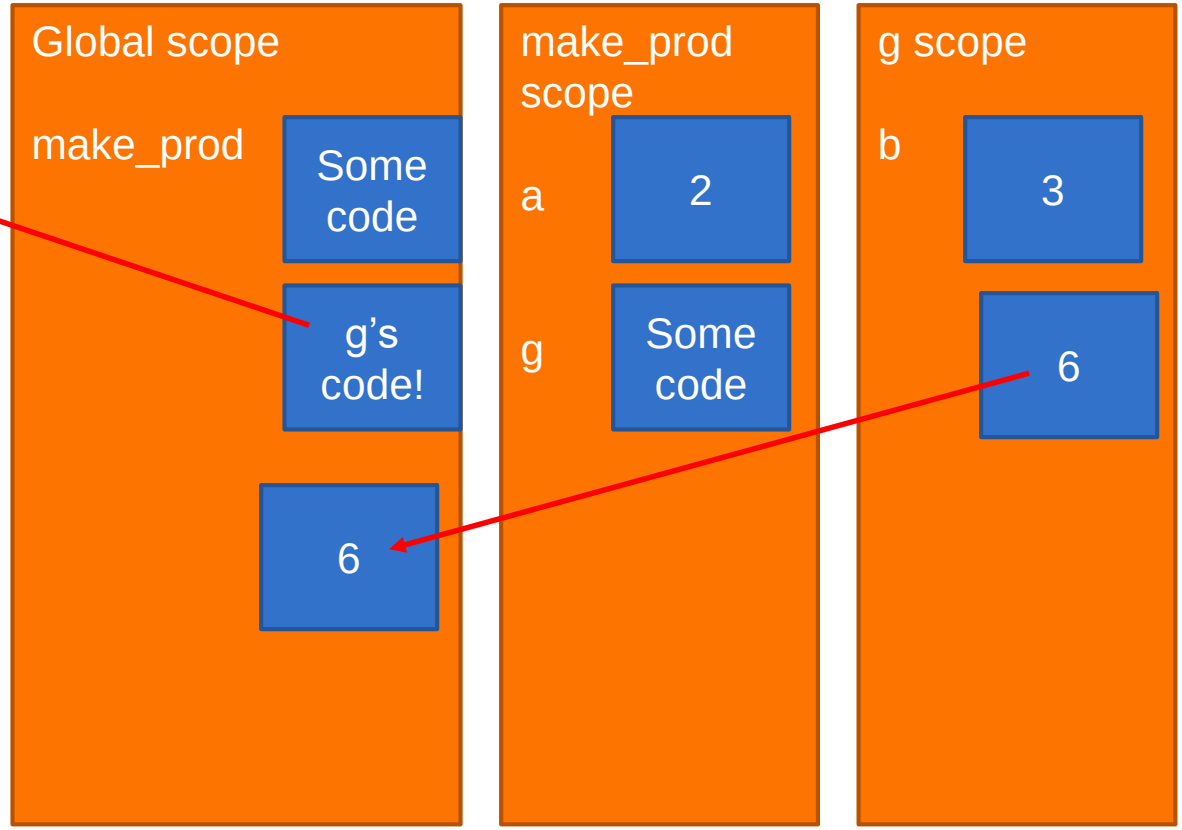
```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```



How does g get value for a?
Interpreter can move up hierarchy of frames to see both b and a values

SCOPE DETAILS FOR WAY 1

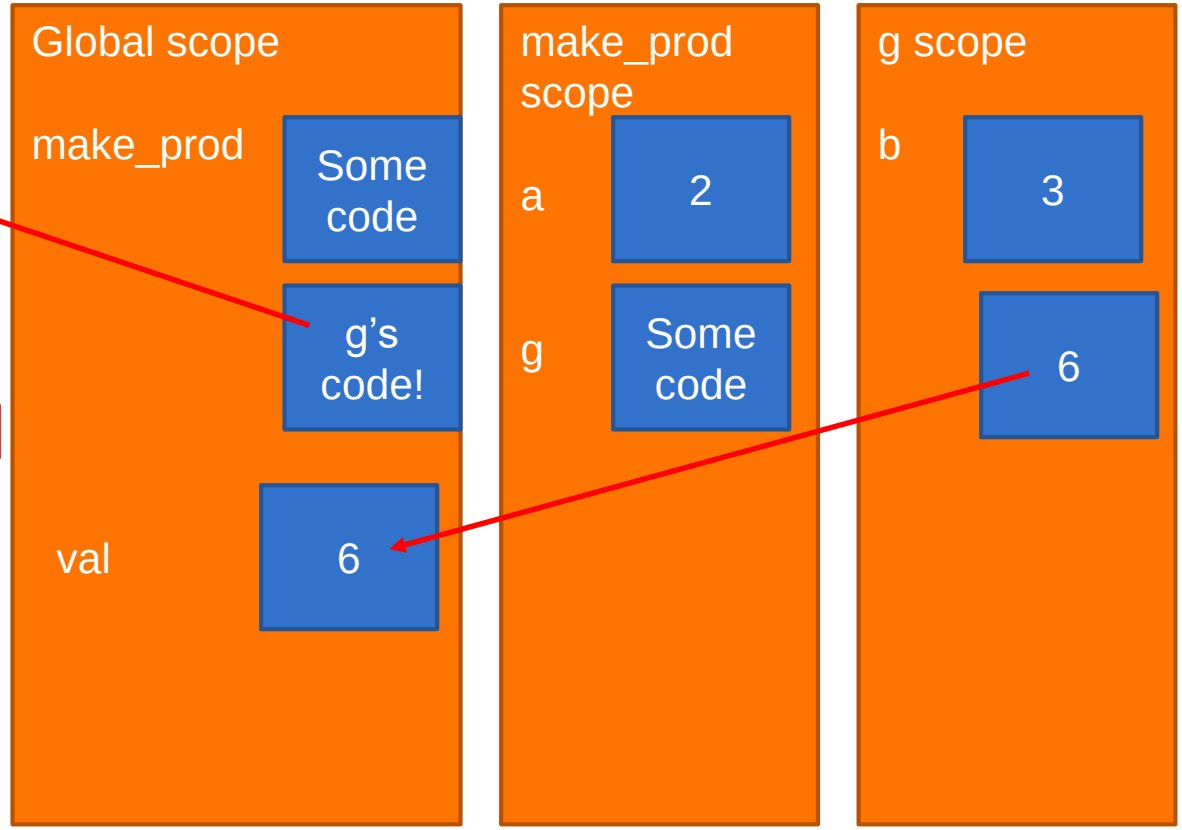
```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```



How does g get value for a?
Interpreter can move up hierarchy of frames to see both b and a values

SCOPE DETAILS FOR WAY 1

```
def make_prod(a):
    def g(b):
        return a*b
    return g

val = make_prod(2)(3)
print(val)
```



How does g get value for a?
Interpreter can move up hierarchy of frames to see both b and a values

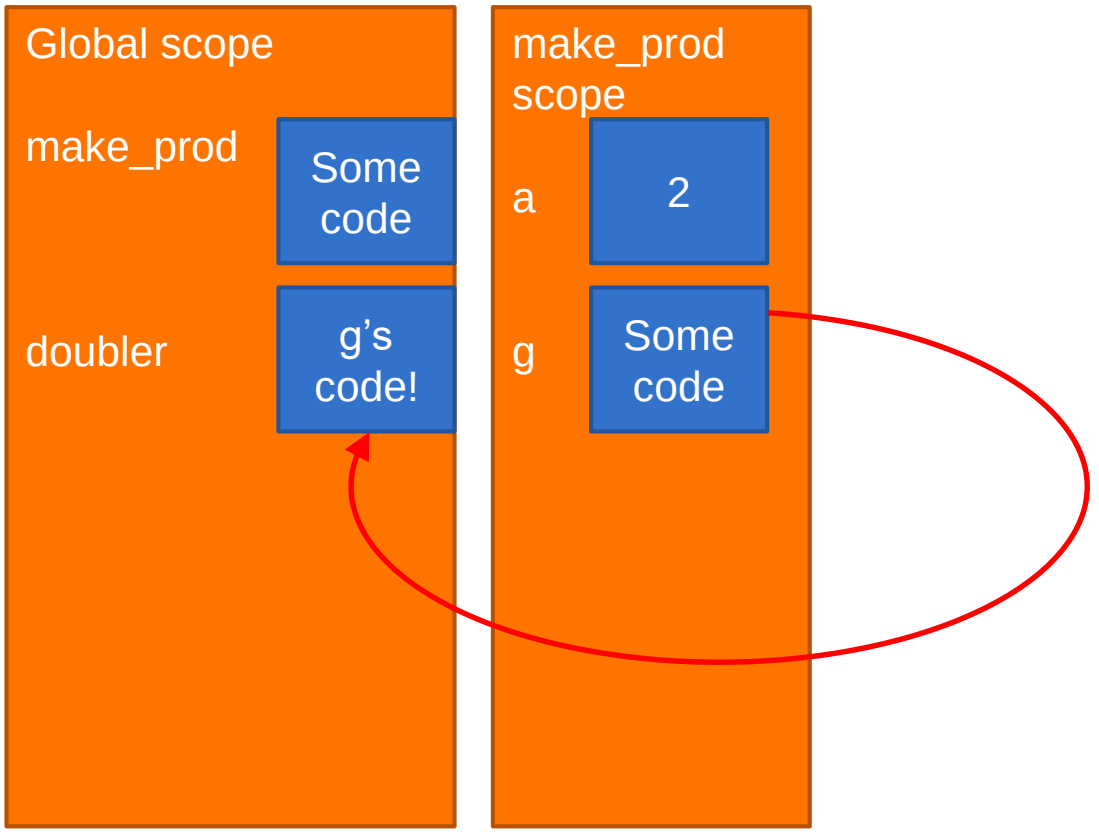
SCOPE DETAILS FOR WAY 2

```
def make_prod(a):
    def g(b):
        return a*b
    return g

doubler = make_prod(2)
val = doubler(3)
print(val)
```

SCOPE DETAILS FOR WAY 2

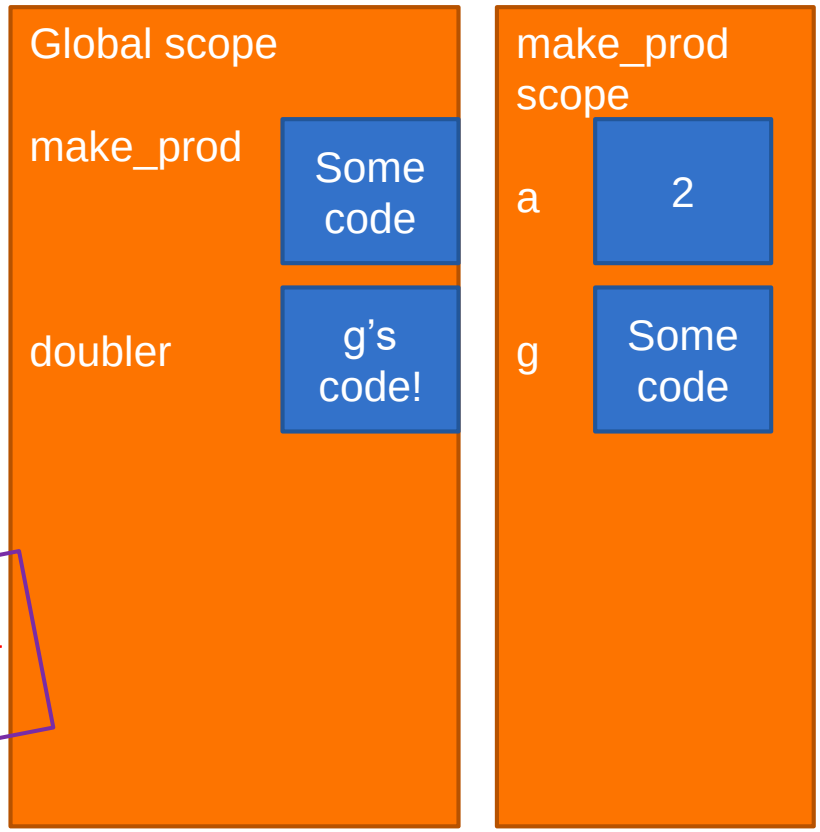
```
def make_prod(a):
    def g(b):
        return a*b
    return g

doubler = make_prod(2)
val = doubler(3)
print(val)
```

g



SCOPE DETAILS FOR WAY 2

```
def make_prod(a):
    def g(b):
        return a*b
    return g

doubler = make_prod(2)
val = doubler(3)
print(val)
```

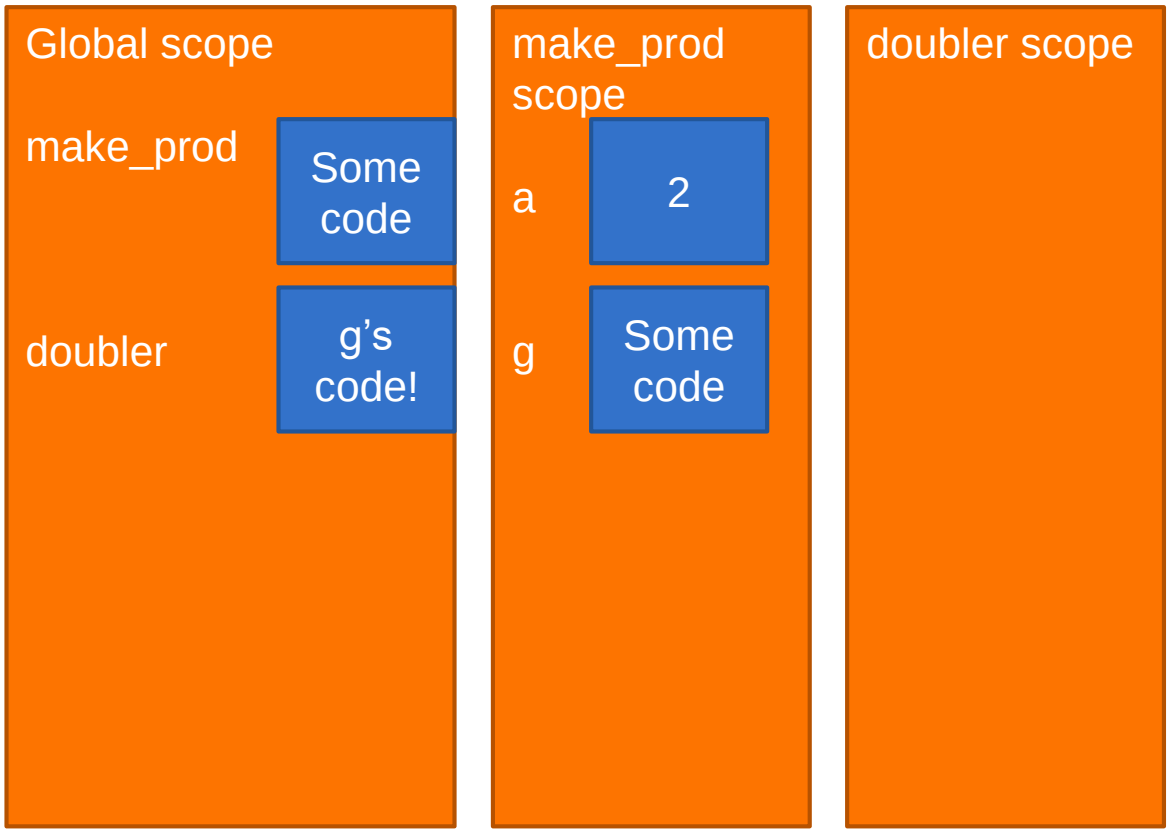


Evaluating `make_prod(2)` has same effect as previous example — doubler is a **function object**

SCOPE DETAILS FOR WAY 2

```
def make_prod(a):
    def g(b):
        return a*b
    return g


doubler = make_prod(2)
val = doubler(3)
print(val)
```

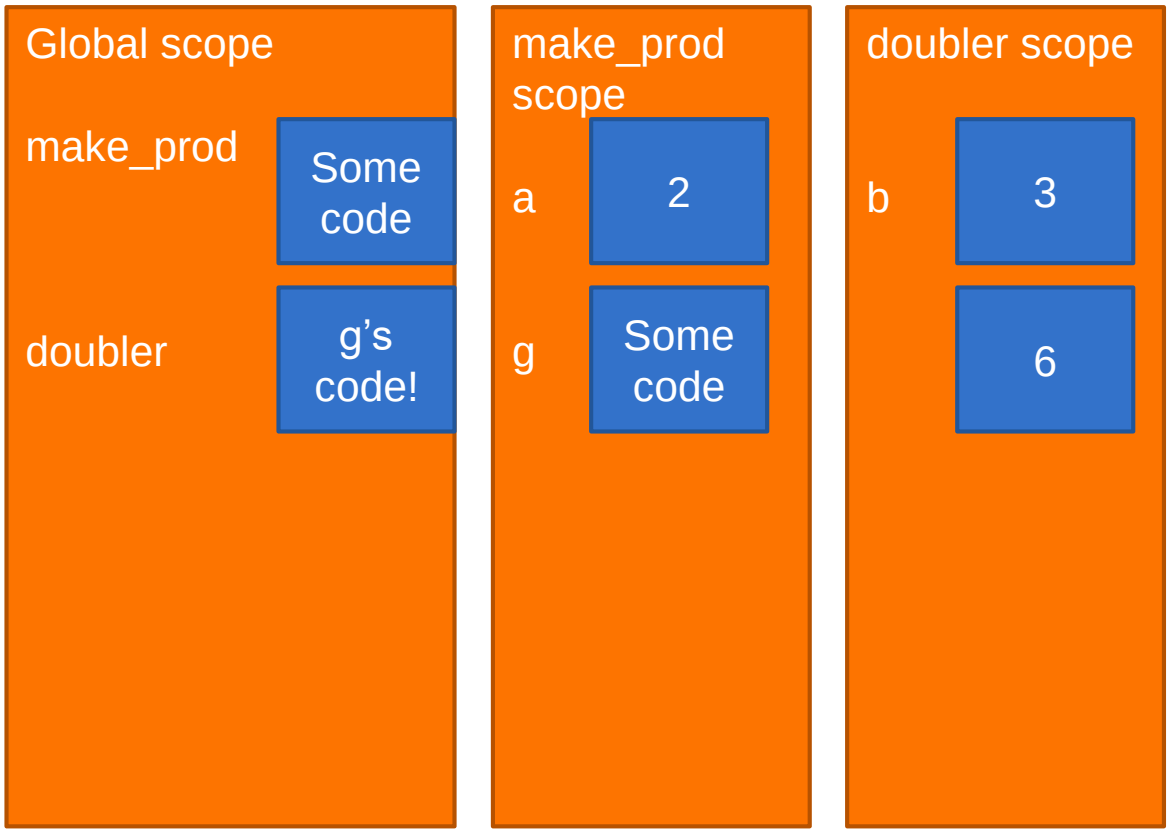
Now invoking g(3)



SCOPE DETAILS FOR WAY 2

```
def make_prod(a):
    def g(b):
        return a*b
    return g

doubler = make_prod(2)
val = doubler(3)
print(val)
```

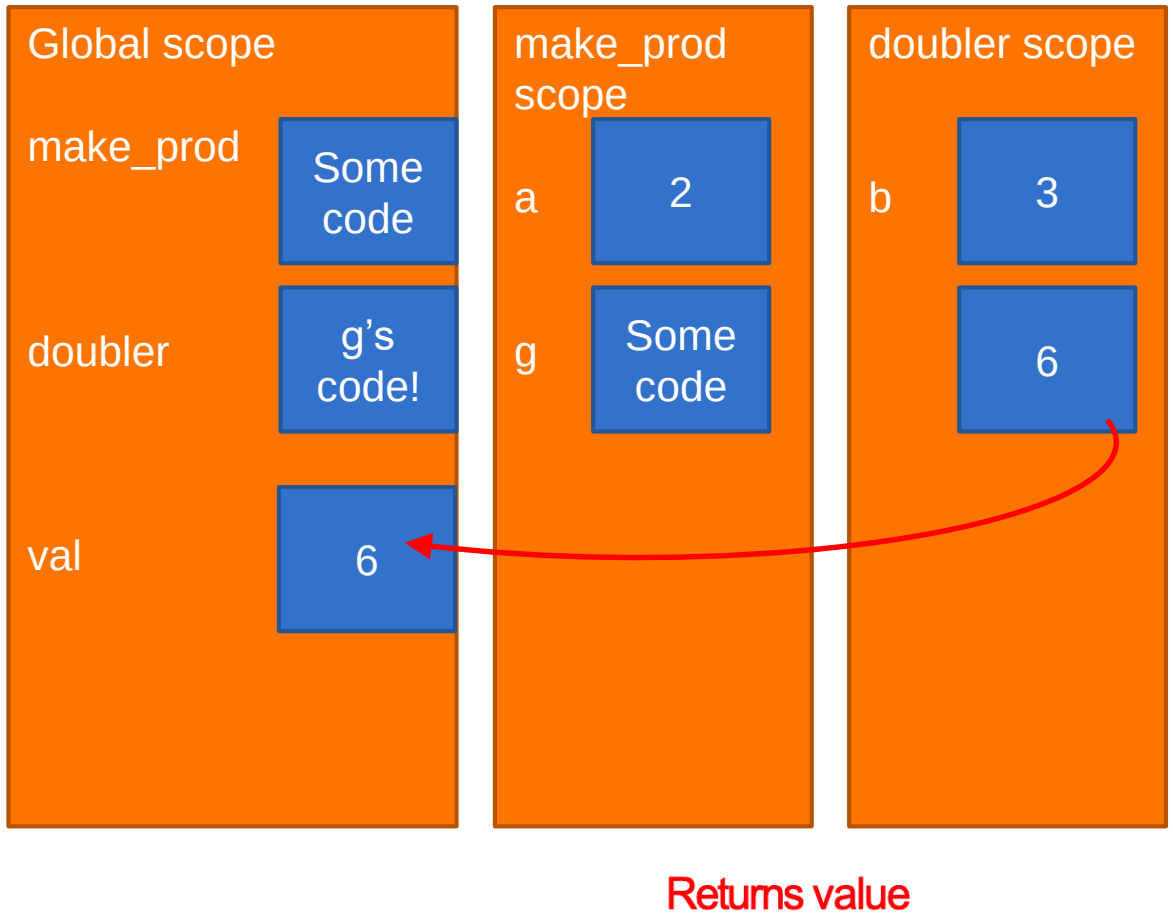
Now invoking g(3)



SCOPE DETAILS FOR WAY 2

```
def make_prod(a):
    def g(b):
        return a*b
    return g

doubler = make_prod(2)
val = doubler(3)
print(val)
```

Now invoking g(3)



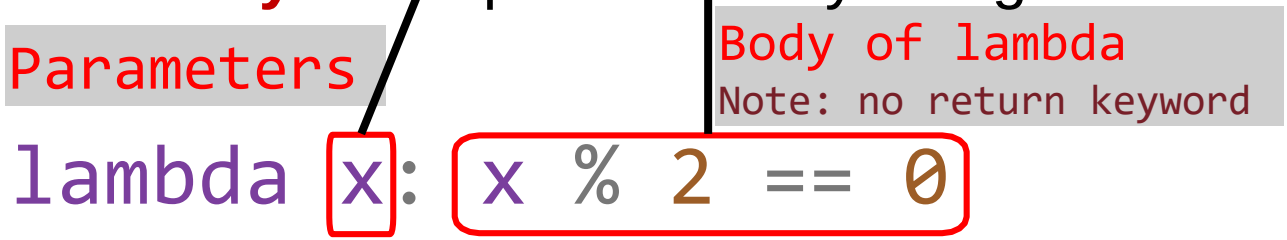
LAMBDA FUNCTIONS

ANONYMOUS FUNCTIONS

- Sometimes don't want to name functions, especially simple ones. This function is a good example:

```
def is_even(x):
    return x % 2 == 0
```

- Can use an **anonymous** procedure by using `lambda`



- `lambda` creates a procedure/function object, but simply does not bind a name to it

ANONYMOUS FUNCTIONS

- Function call with a named function:

```
apply(is_even, 10)
```

- Function call with an anonymous function as parameter:

```
apply( lambda x: x%2 == 0 , 10 )
```

- `lambda` function is **one-time use**. It can't be reused because it has no name!

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

Global Environment

`do_twice: FunctionType`

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

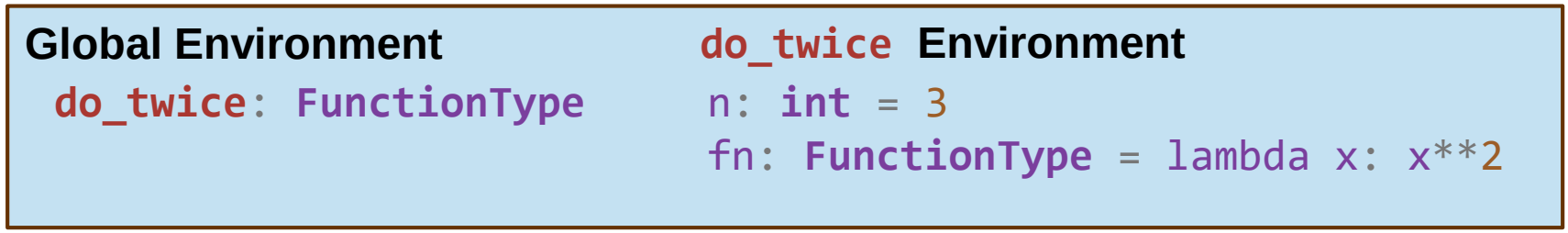
Global Environment

`do_twice: FunctionType`

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```



YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

Global Environment

```
do_twice: FunctionType
```

do_twice Environment

```
n: int = 3
fn: FunctionType = lambda x: x**2
```

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

Global Environment

```
do_twice: FunctionType
```

do_twice Environment

```
n: int = 3
fn: FunctionType = lambda x: x**2
```

(lambda x: x2) Environment**

```
x: Any = fn(n)
```

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

Global Environment

```
do_twice: FunctionType
```

do_twice Environment

```
n: int = 3
fn: FunctionType = lambda x: x**2
```

(lambda x: x2) Environment**

```
x: Any = fn(n)
```

(lambda x: x2) Environment**

```
x: int = n = 3
```

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

Global Environment

```
do_twice: FunctionType
```

do_twice Environment

```
n: int = 3
fn: FunctionType = lambda x: x**2
```

(lambda x: x2) Environment**

```
x: Any = fn(n)
```

(lambda x: x2) Environment**

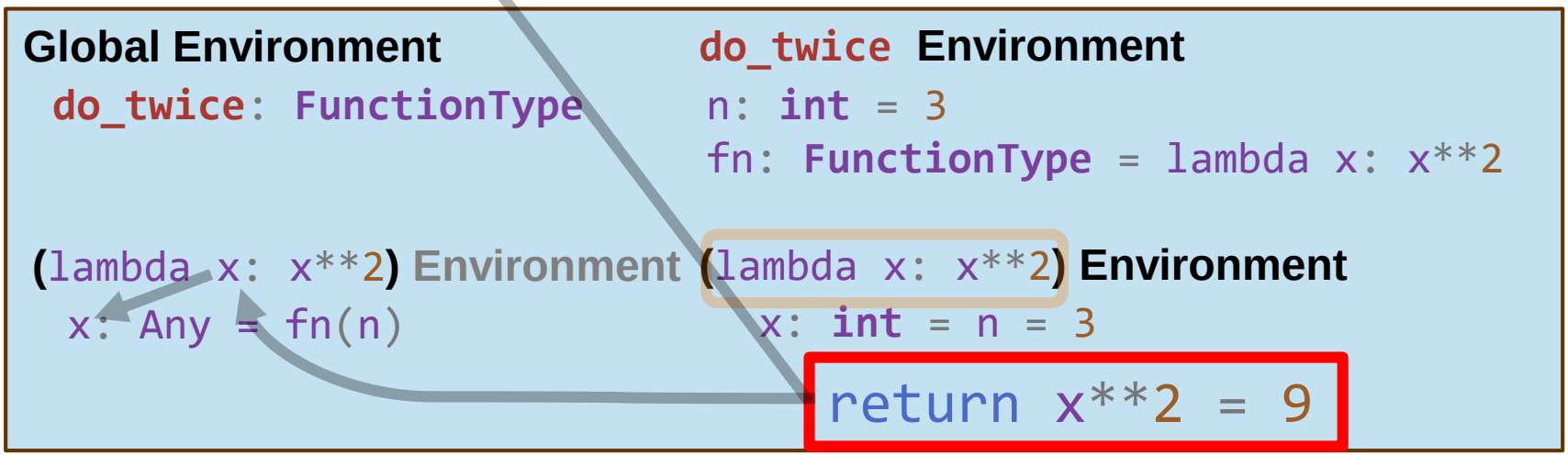
```
x: int = n = 3
```

```
return x**2 = 9
```

YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```



YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```

Global Environment

```
do_twice: FunctionType
```

do_twice Environment

```
n: int = 3
fn: FunctionType = lambda x: x**2
```

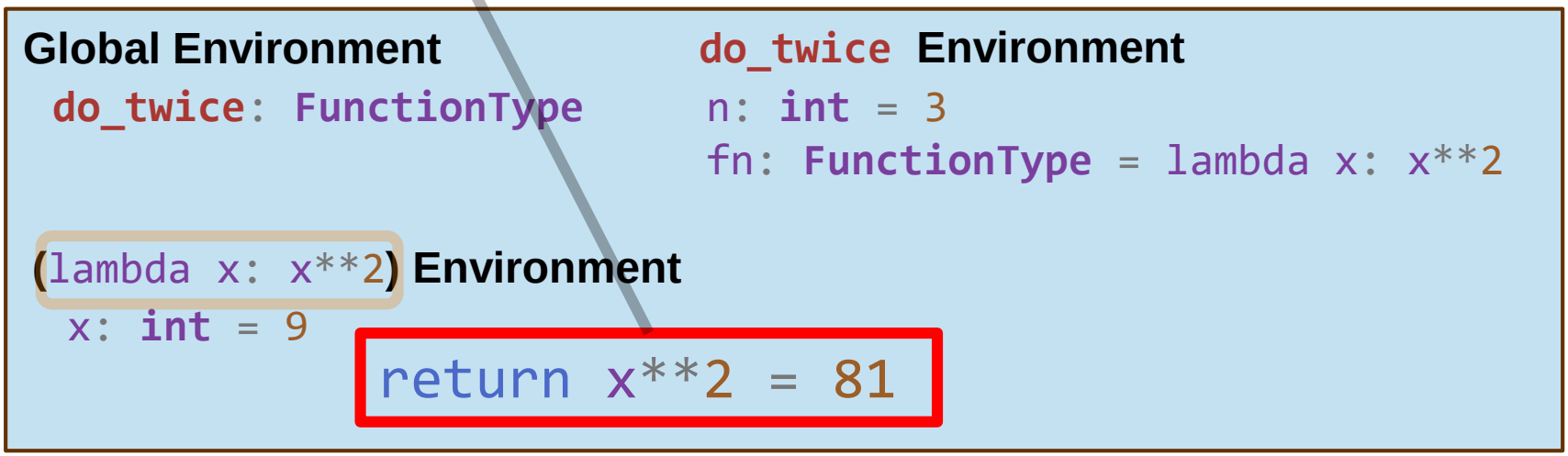
(lambda x: x2) Environment**

```
x: int = 9
```

YOU TRY IT!

- What does this print?

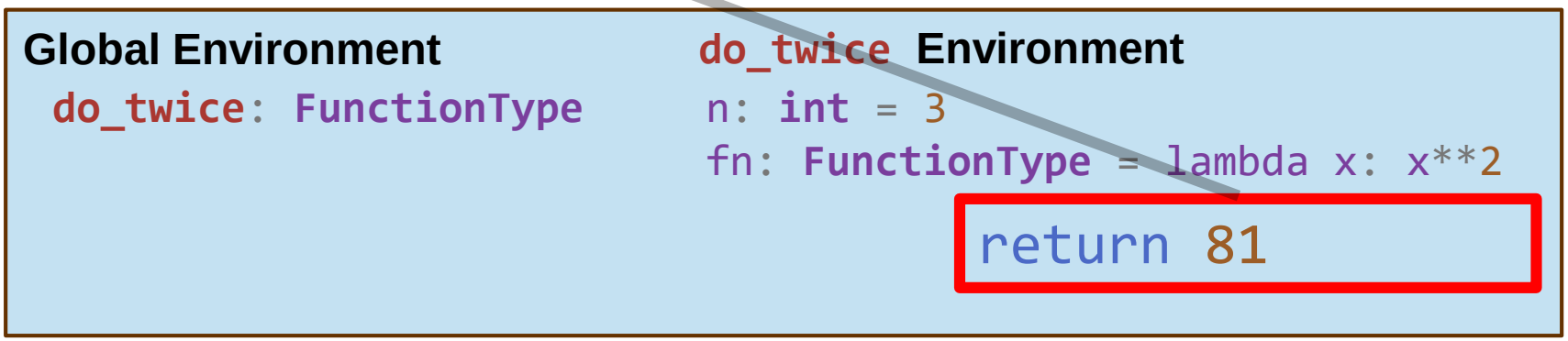
```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```



YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))

print(do_twice(3, lambda x: x**2))
```



YOU TRY IT!

- What does this print?

```
def do_twice(n, fn):
    return fn(fn(n))
```

```
print(do_twice(3, lambda x: x**2))
```

Global Environment

```
do_twice: FunctionType
```

print Environment

```
*values = do_twice(...) = 81
```

Output:

81

SUMMARY

- Functions are first class objects
  - They have a **type**
  - They can be **assigned as a value** bound to a name
  - They can be used as an **argument** to another procedure
  - They can be **returned** as a value from another procedure
- Have to be careful about environments
  - Main program runs in the global environment
  - Function calls each get a new temporary environment
- This enables the creation of concise, easily read code

TUPLES

A NEW DATA TYPE

- Have seen scalar types: `int`, `float`, `bool`
- Have seen one compound type: `string`
- Want to introduce more general **compound data types**
  - Indexed sequences of elements, which could themselves be compound structures
  - **Tuples** – immutable
  - **Lists** – mutable

TUPLES

Remember strings?

- **Indexable ordered sequence** of objects
  - Objects can be any type – int, string, tuple, tuple of tuples, …
- Cannot change element values, **immutable**

```
tuple_empty = ()
tuple_short = (2,)

hello_tuple = (2, "stu", 3)
hello_tuple[0]  # = 2
hello_tuple[1:3]  # = ("stu", 3)

(2, "stu", 3) + (5, 6)  # = (2, "stu", 3, 5, 6)
(1, 2) * 2  # (1, 2, 1, 2)

len(hello_tuple)  # = 3
max((3, 5, 0))  # = 5
hello_tuple[1] = 4  # TypeError: 'tuple' object does not support item assignment
```

Empty tuple

One element with a **comma** is a tuple `(2,)`
What about without a **comma?** `(2)`

Multiple elements in tuple separated by commas

Indexing (starts at 0)

Slice tuple

`min`, **`sum`**, **`sorted`**, …

✗ Immutable Types does not support modification.

INDICES AND SLICING

```
seq = (2, "a", 4, (1, 2))
 Index  0    1    2     3
```

```
print(len(seq))  # 4
print(seq[3])  # (1, 2)
print(seq[-1])  # (1, 2)
print(seq[3][0]) # 1
print(seq[4])  # IndexError
```

An element of a sequence is at an **index**, indices start at 0

```
print(seq[1])  # a
print(seq[-2:])  # (4, (1, 2))
print(seq[1:4:2])  # ('a', (1, 2))
print(seq[:-1])  # (2, 'a', 4)
print(seq[1:3])  # ('a', 4)
```

Slices extract subsequences
Indices evaluated from left to right

```
for e in seq:  # 2
    print(e)   # a
               # 4
               # (1, 2)
```

Iterating over sequences

TUPLES

- Conveniently used to **swap** variable values

```
x = 1
y = 2
x = y
y = x
```

```
x = 1
y = 2
temp = x
x = y
y = temp
```

```
x = 1
y = 2
(x, y) = (y, x)
```

Unpacking to x, y

Pack y, x into a tuple

TUPLES

- Used to return more than one value from a function

```
def quotient_and_remainder(x, y):
    q = x // y
    r = x % y
    return (q, r)
```

One Object! (with 2 elements)

```
both = quotient_and_remainder(10, 3)
```

(3, 1)

(3, 1)

```
(quot, rem) = quotient_and_remainder(5, 2)
```

(2, 1)

2

1

BIG IDEA

- Returning one **object** (a tuple) allows you to return multiple **values** (tuple elements)

YOU TRY IT!

- Write a function that meets these specs:
- Hint: remember how to check if a character is in a string?

```
def char_counts(s):
    """
    s is a string of lowercase chars
    Return a tuple where the first element is the number of
    vowels in s and the second element is the number of
    consonants in s
    """
    # ... your code ...
```

VARIABLE NUMBER of ARGUMENTS

- Python has some built-in functions that take variable number of arguments, e.g, `min`
- Python allows a programmer to have same capability, using *** notation**

```
def mean(*args):
    tot = 0
    for a in args:
        tot += a
    return tot / len(args)
```

Pack into a Tuple
(1, 2, 3, 4, 5, 6)

- `args` is bound to a **tuple of the supplied values**
- Example:

```
mean(1, 2, 3, 4, 5, 6)
```

TUPLES

- Parentheses are optional (except in the cases of the empty tuple or syntactic ambiguity)
- It is the comma which makes a tuple, not the parentheses.
- These are equivalent:
  - `(x, y) = (1, 2)`
  - `(x, y) = 1, 2`
  - `x, y = (1, 2)`
  - `x, y = 1, 2`

LISTS

LISTS

- **Indexable ordered sequence** of objects
  - Usually homogeneous (i.e., all integers, all strings, all lists)
  - But can contain mixed types (not common)
- Denoted by **square brackets**, [ ]
- **Mutable**, this means you can change values of specific elements of list

Notes: Tuples were: ()

Remember **tuples** are **immutable** -- you cannot change element values.
Lists are **mutable**, you can change them directly.

INDICES and ORDERING

Remember strings and tuples?

```
empty_list = []
short_list = [2]
hello_list = [2, "a", 4, [1, 2]]
hello_list[0]  # 2
hello_list[3]  # [1, 2]
hello_list[4]  # IndexError: list index out of range
hello_list[1:2]  # = ["a"]
hello_list[1:3]  # = ["a",4]

[1, 2] + [3, 4]  # [1, 2, 3, 4]
[1, 2] * 2  # [1, 2, 1, 2]

len(hello_list)  # 4
max([3, 5, 0])  # 5
hello_list[0] = 3
```

← Empty list

✔ Mutable (supports modification)

ITERATING OVER a LIST

- Like string or tuple, can iterate over elements of list directly

```
L = [3, 5, 0]
total = 0
for v in L:
    total += v
```

YOU TRY IT!

- Write a function that meets these specs:

```
def sum_and_prod(L):
    """
    L is a list of numbers
    Return a tuple where the first value is the
    sum of all elements in L and the second value
    is the product of all elements in L
    """
    # ... your code ...
```